Hotel Booking Cancellation Prediction

End-to-End Machine Learning Project — Technical & User Documentation

NTI Training Program — Machine Learning for Data Analysis

Best Model:	CatBoost Classifier
Test Accuracy:	85.27%
ROC AUC Score:	91.98%
Deployment:	Streamlit Web Application

Prepared as project documentation — Data Science / ML Track

Table of Contents

1. Project Overview

This project delivers a complete, end-to-end Machine Learning pipeline that predicts whether a hotel reservation will be **canceled** before the guest's arrival date. It was built as a graduation project for the NTI (National Telecommunication Institute) *Machine Learning for Data Analysis* track, and walks through every stage of a real data-science workflow: data understanding, cleaning, encoding, exploratory analysis, feature engineering, model comparison, hyperparameter tuning, evaluation, and finally deployment as an interactive web application built with Streamlit.

The target column is **is_canceled**, a binary label where 0 means the booking was honored and 1 means it was canceled. Being able to flag high-risk reservations in advance gives a hotel's revenue and operations teams a practical tool: overbooking strategy can be adjusted, deposits can be requested for risky bookings, and staffing/inventory planning becomes more reliable.

Core objectives of the project:

- Predict the probability that an individual booking will be canceled.
- Understand the behavioral and booking-related factors that drive cancellations.
- Systematically compare a wide range of classification algorithms rather than relying on a single model.
- Package the winning model into an interactive application usable by non-technical staff.

2. Dataset Description

The project uses the well-known **Hotel Booking Demand** dataset, which combines records from a City Hotel and a Resort Hotel. Each row represents one booking and carries information across several categories:

- **Booking information** — lead time, arrival date (year/month/week/day), length of stay (weekday/weekend nights).
- **Customer information** — customer type, repeated-guest flag, number of adults/children/babies, country of origin.
- **Hotel information** — hotel type (City / Resort), meal plan.
- **Room information** — reserved room type versus the room type actually assigned.
- **Payment information** — average daily rate (ADR), deposit type.
- **Reservation history** — previous cancellations, previous non-canceled bookings, booking changes, days on the waiting list.

As shown in the app's own summary panel, the modeling dataset contains roughly **74,361 records** after cleaning, with **is_canceled** as the classification target.

3. Methodology — Cleaning, Encoding & Scaling

3.1 Data Cleaning

Missing values were treated differently depending on the data type in order to avoid discarding rows: **numerical features** were filled using median imputation (robust to outliers), while **categorical features**

were filled using most-frequent (mode) imputation. This preserves the full size of the dataset instead of dropping incomplete rows, which matters for a dataset of this scale.

3.2 Smart Encoding Strategy

Rather than applying one encoding scheme to every categorical column, the encoding method was chosen based on each feature's nature:

- **Label Encoding** — applied to hotel, meal, market segment, distribution channel, reserved/assigned room type, deposit type, and customer type.
- **Ordinal Encoding** — applied to arrival month specifically, since calendar months have a natural chronological order that label encoding alone would not preserve consistently.
- **Frequency Encoding** — applied to country, which has a very large number of unique values; this avoids the huge sparse vectors that one-hot encoding would create while still preserving useful information about how common each country is.

3.3 Feature Scaling

A **StandardScaler** was fit on the numerical columns only (categorical/encoded columns were left untouched). Scaling speeds up convergence for gradient-based models and improves distance-based calculations, which benefits algorithms such as KNN and Logistic Regression in the comparison.

4. Exploratory Data Analysis — Key Insights

A thorough visual exploration was carried out before modeling. The charts below (taken directly from the project's EDA stage) summarize the most decision-relevant patterns in the data.

4.1 Overall Distributions

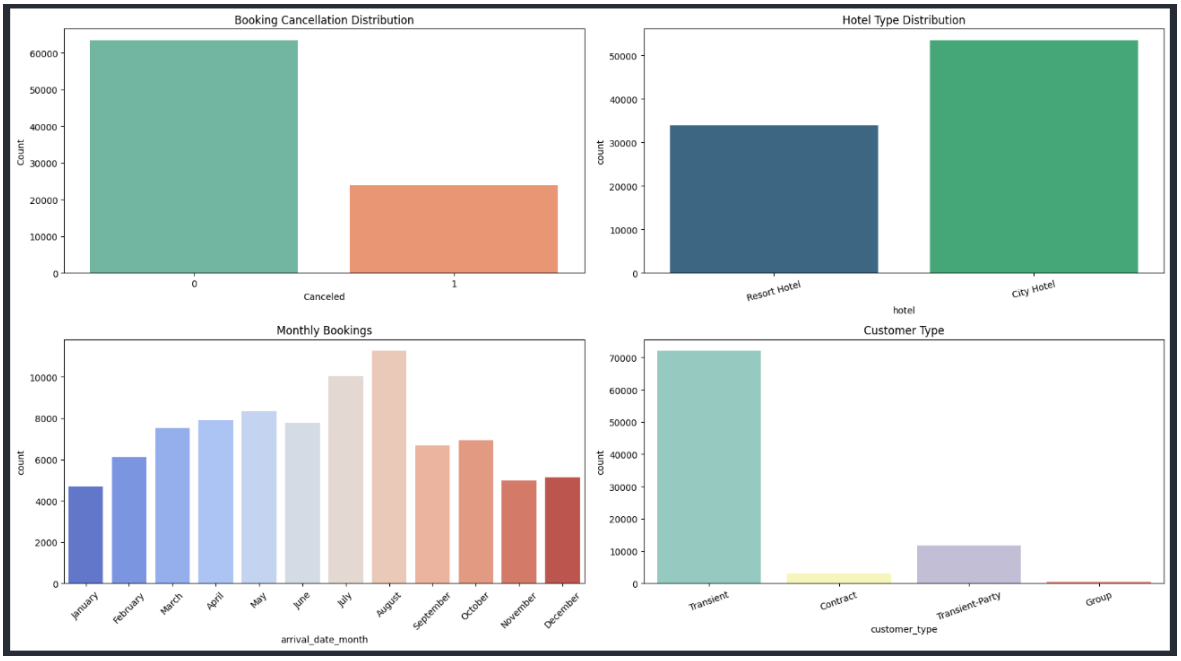

Figure 1 — Cancellation distribution, hotel type split, monthly booking volume, and customer type breakdown.

Insight:

Around a quarter of all bookings end in cancellation, so the target classes are imbalanced (roughly 3-to-1 in favor of non-canceled bookings) — this is why the project relies on F1 score, recall, and ROC AUC rather than accuracy alone. City Hotel bookings outnumber Resort Hotel bookings by a wide margin, booking volume clearly peaks in the summer months (July–August), and the overwhelming majority of guests are classified as **Transient** customers rather than groups or contracts.

4.2 Cancellation Rate by Month

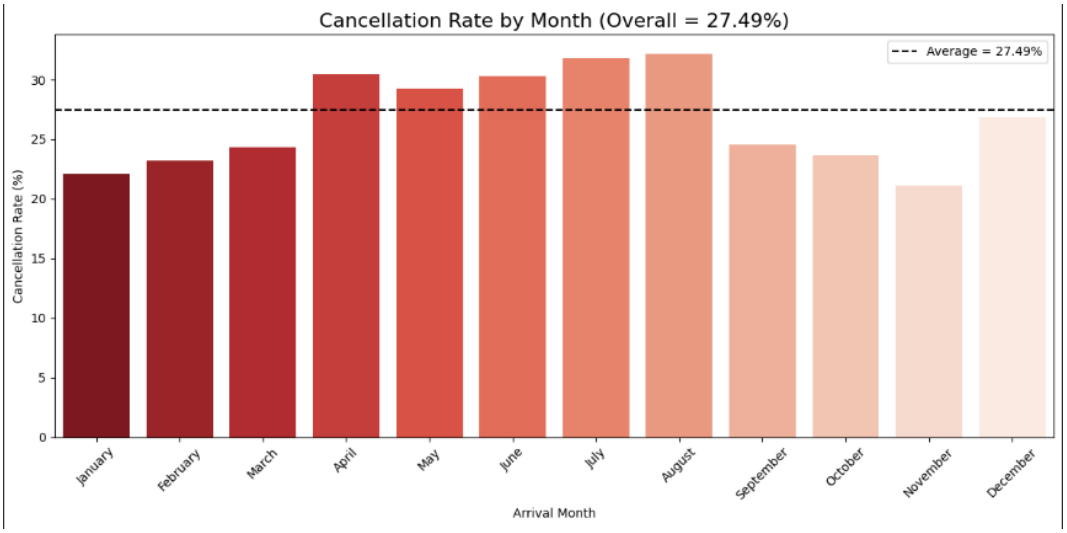

Figure 2 — Monthly cancellation rate versus the overall average of 27.49%.

Insight:

Cancellation rates are not flat across the year — they rise above the 27.49% average from April through August, peaking in **July and August** at above 30%, then drop below average in the autumn and winter months (especially November). This seasonal pattern suggests that high-demand summer bookings are also higher-risk, possibly because guests book further ahead and have more time to change plans.

4.3 Cancellation Rate by Lead Time

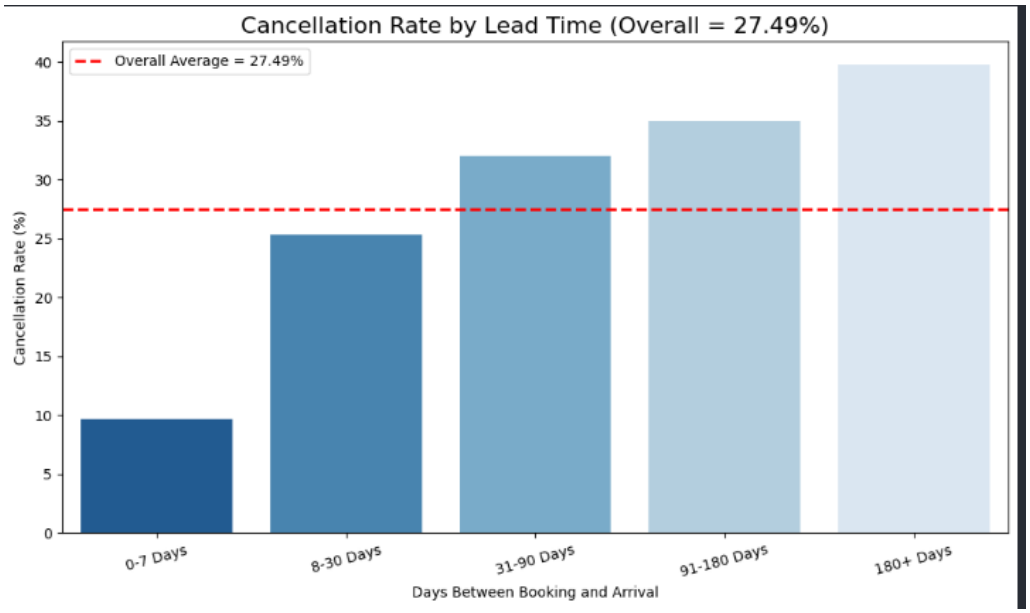

Figure 3 — Cancellation rate grouped into lead-time buckets (days between booking and arrival).

Insight:

This is one of the strongest and clearest patterns in the whole dataset: cancellation risk increases almost monotonically with lead time. Bookings made 0–7 days before arrival cancel less than 10% of the time, while bookings made **more than 180 days** in advance cancel nearly **40%** of the time. This makes lead time one of the most powerful predictive features and a natural candidate for risk-based deposit or confirmation policies on long-lead bookings.

4.4 Cancellation Rate by Country

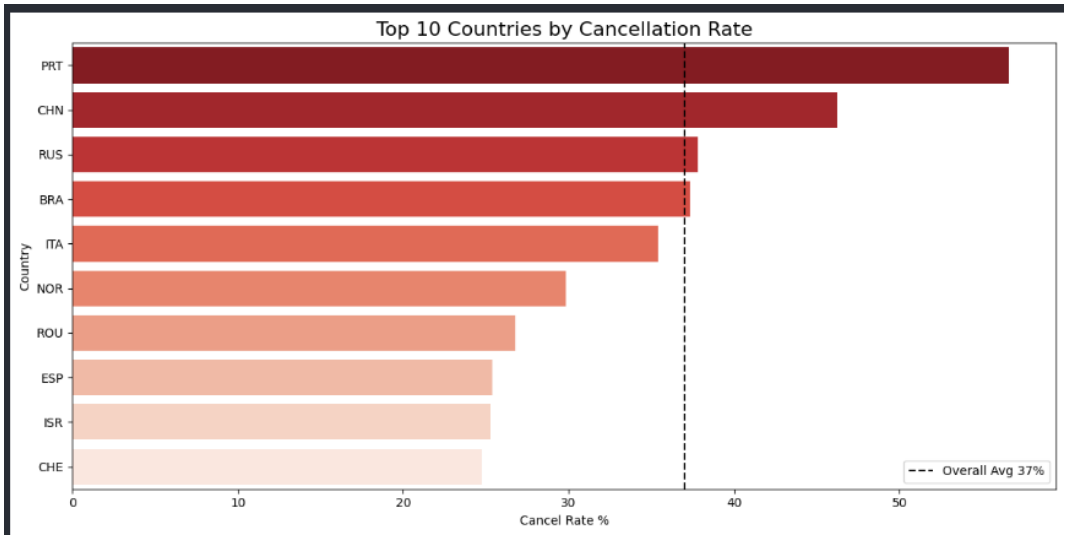

Figure 4 — Top 10 countries by cancellation rate (countries with at least 500 bookings).

Insight:

Guests from **Portugal (PRT)** and **China (CHN)** show cancellation rates far above the 37% average for this subset — Portugal alone approaches 57%. Russia, Brazil, and Italy also sit above average, while countries such as Switzerland, Israel, and Spain are noticeably more reliable. Country of origin is therefore a meaningful risk signal and is used in the model through frequency encoding.

4.5 Outlier Inspection

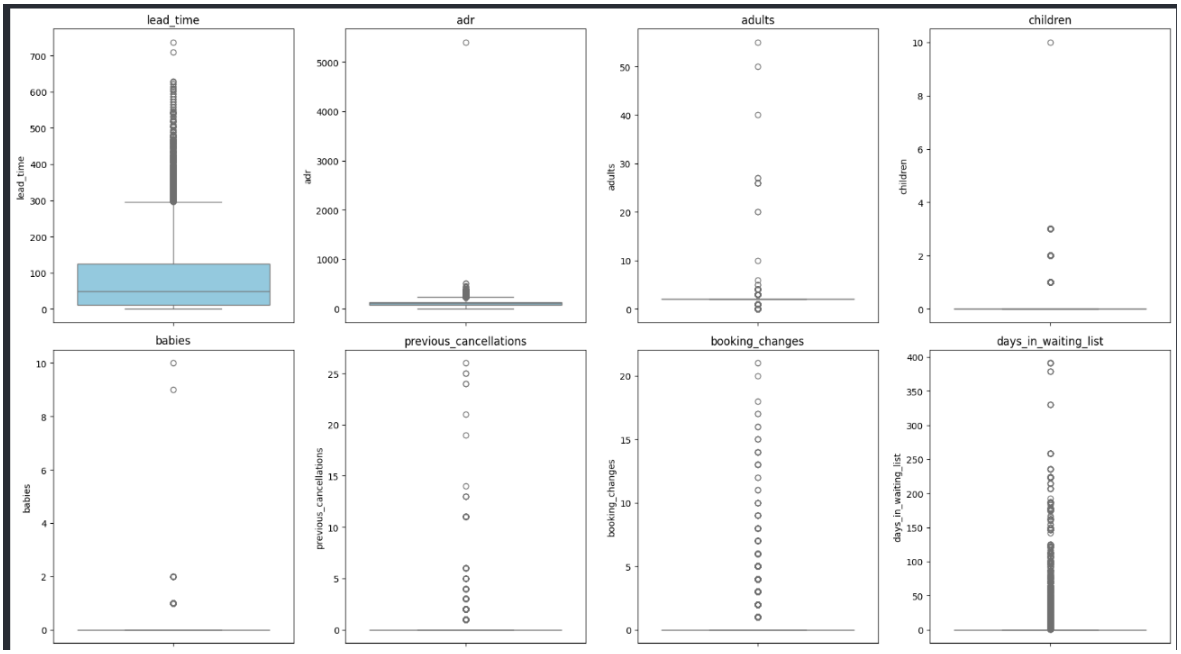

Figure 5 — Boxplots of key numerical features (lead_time, adr, adults, children, babies, previous_cancellations, booking_changes, days_in_waiting_list).

Insight:

Nearly every numerical feature carries a long right tail of outliers — a small number of bookings with extremely long lead times, unusually high ADR (with one extreme value near 5,000+), very large party sizes, or very long waiting-list durations. These were kept rather than removed (tree-based models such as CatBoost and XGBoost are naturally robust to them), but they are worth flagging for anyone re-using this data with distance-based or linear models, which are more sensitive to extreme values.

4.6 Correlation with the Target

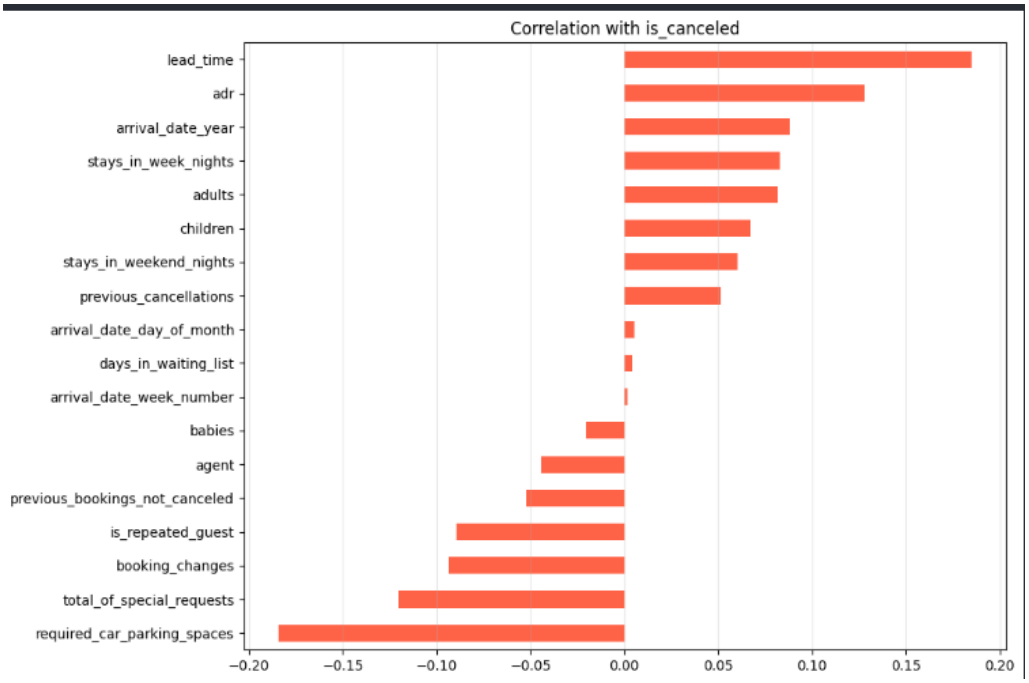

Figure 6 — Linear correlation of each numerical feature with is_canceled.

Insight:

Lead time and **ADR** show the strongest positive correlation with cancellation, confirming the lead-time pattern seen above and suggesting that pricier rooms are somewhat more likely to be canceled. On the other side, **required car parking spaces**, **total special requests**, and **booking changes** show the strongest negative correlation — guests who request parking, make special requests, or actively modify their booking are behaviorally more committed and far less likely to cancel. These correlations, while modest individually (as expected for a multi-factor outcome like cancellation), validate the feature set chosen for modeling.

5. Feature Engineering & Model Building

After encoding and scaling, a feature-importance-driven selection process was used to narrow the full feature set down to the **top 20 most informative features**, which were then used for the final model. Leading features include Country, Lead Time, ADR, Market Segment, Assigned Room Type, Customer Type, Required Car Parking Spaces, Total Special Requests, Booking Changes, and Arrival Year.

Thirteen algorithms were trained and evaluated on an identical train/test split for a fair comparison: Logistic Regression, KNN, Decision Tree, Random Forest, Extra Trees, AdaBoost, Gradient Boosting, Gaussian Naive Bayes, XGBoost, LightGBM, CatBoost, a Soft Voting ensemble, and a Stacking ensemble. Each was scored on train accuracy, test accuracy, cross-validation score, precision, recall, F1 score, ROC AUC, and training time — a much fuller picture than accuracy alone would give.

6. Model Comparison & Results

The table below reproduces the actual comparison results generated in the notebook, sorted by test accuracy:

Model	Train Acc.	Test Acc.	CV Score	Precision	Recall	F1	ROC AUC
CatBoost	**0.877**	**0.854**	**0.854**	**0.758**	**0.687**	**0.721**	**0.921**
XGBoost	0.880	0.853	0.853	0.755	0.690	0.721	0.919
Random Forest	0.998	0.850	0.850	0.766	0.655	0.706	0.912
LightGBM	0.858	0.848	0.851	0.753	0.667	0.707	0.916
Extra Trees	0.998	0.846	0.845	0.759	0.646	0.698	0.899
Gradient Boosting	0.834	0.831	0.832	0.753	0.571	0.650	0.886
Decision Tree	0.998	0.797	0.795	0.626	0.646	0.636	0.751
KNN	0.847	0.774	0.773	0.601	0.527	0.562	0.788
Logistic Regression	0.775	0.772	0.775	0.644	0.378	0.477	0.811
AdaBoost	0.763	0.765	0.795	0.716	0.240	0.359	0.830
Gaussian NB	0.403	0.403	0.403	0.312	0.972	0.472	0.741

CatBoost was selected as the final model: it achieved the top test accuracy and F1 score while keeping cross-validation score almost identical to test accuracy (indicating low overfitting compared to Random Forest/Extra Trees, whose ~99.8% train accuracy signals heavy overfitting). CatBoost also handles categorical-style features natively and offers a strong ROC AUC of 0.921.

RandomizedSearchCV was then used to tune CatBoost's hyperparameters (chosen over an exhaustive grid search for its ability to explore a wider parameter space in less time), and the top-20 feature subset was locked in after comparing multiple feature-count options. The tuned model's final held-out performance was:

Metric	Value
Accuracy	85.27%
Precision	75.43%

Recall	68.87%
F1 Score	71.99%
ROC AUC	91.98%

On the held-out test set (17,480 bookings), the model correctly identifies 91% of non-canceled and 75% of canceled bookings at the precision level, with a recall of ~69% on the cancellation class — meaning roughly 7 in 10 cancellations are caught in advance, which is highly actionable for revenue management even though a share of cancellations still go undetected.

7. Streamlit Application — Code Walkthrough

The trained pipeline is deployed as an interactive Streamlit application (`app.py`). The script is organized into clear, sequential blocks:

7.1 Page Setup & Sidebar

The app configures a wide layout with an expanded sidebar via `st.set_page_config`. The sidebar acts as a project fact-sheet: it names the best model (CatBoost), displays headline metrics (accuracy and ROC AUC) using `st.sidebar.metric`, and summarizes the dataset (record count and prediction target).

7.2 Loading Saved Artifacts

A single cached function, `load_files()`, decorated with `@st.cache_resource`, loads every artifact produced at the end of training: the final CatBoost model, the numeric and categorical imputers, the label encoders dictionary, the hotel encoder, the month (ordinal) encoder, the country frequency map, the scaler, and the list of the 20 selected best features. Caching means these objects are loaded from disk only once per session rather than on every user interaction, which keeps the app responsive.

7.3 User Input Form

Inputs are grouped into three headed sections using Streamlit columns for a clean two/three-column layout:

- **Hotel Information** — hotel type, market segment, distribution channel, deposit type, customer type, reserved/assigned room type, and country (free-text, e.g. "PRT").
- **Booking Information** — lead time, arrival year/week/day, weekend/week nights, previous cancellations, booking changes, and ADR.
- **Additional Information** — required parking spaces, total special requests, and agent ID.

A few fields that the interface does not expose to the user (meal plan and arrival month) are hard-coded to sensible defaults ("BB" and "January"), as are adults/children/babies, repeated-guest flag, previous non-canceled bookings, and days on the waiting list — see Section 10 for the implications of this design choice.

7.4 Building the Feature Row & Preprocessing

The collected inputs are assembled into a single-row pandas DataFrame with all 27 raw features the training pipeline expects. The exact same preprocessing sequence used during training is then replayed on this single row:

- Missing-value imputation via the saved numeric and categorical imputers.
- Encoding — hotel via its dedicated label encoder, arrival month via the ordinal month encoder, country via the frequency map (unseen countries default to a frequency of 0), and the remaining categorical columns via their respective saved label encoders.
- Scaling of all numerical columns using the saved StandardScaler.
- Feature selection — the DataFrame is finally sliced down to just the 20 columns in best_features, matching what the model was trained on.

7.5 Prediction & Result Display

The processed row is passed to `final_model.predict()` and `predict_proba()`. The result is shown with a colored status message (a red error banner for a predicted cancellation, a green success banner otherwise), a confidence metric, a progress bar, and the full probability breakdown for both classes.

8. User Guide — How to Use the Application

8.1 Running the App

From a terminal, inside the project folder containing `app.py` and all the saved `.pkl` artifact files (final_model, numeric_imputer, categorical_imputer, label_encoders, hotel_encoder, month_encoder, country_frequency, scaler, best_features), run:

```
pip install streamlit pandas numpy scikit-learn catboost
```

```
streamlit run app.py
```

Streamlit will open the app automatically in your default browser (typically at http://localhost:8501).

8.2 Entering a Booking

- **Step 1 — Hotel Information:** choose the hotel type, market segment, distribution channel, deposit type, customer type, reserved and assigned room type, and type in the guest's country code (e.g. PRT, GBR, USA).
- **Step 2 — Booking Information:** set the lead time in days, the arrival year/week/day, the number of weekend and week nights, any previous cancellations, booking changes, and the average daily rate (ADR).
- **Step 3 — Additional Information:** set required parking spaces, total special requests, and the travel agent ID (0 if the booking has no agent).

8.3 Reading the Result

As soon as the inputs are set, the app recalculates and displays the prediction below the form:

- A red **"Booking Will Be Cancelled"** banner signals a high predicted cancellation risk.
- A green **"Booking Will Not Be Cancelled"** banner signals a low predicted risk.
- The **Confidence** metric and progress bar show how strongly the model leans toward its predicted class.
- The **Prediction Probabilities** panel shows the exact percentage split between "Not Cancelled" and "Cancelled".

Practical tip: try adjusting lead time or country while keeping everything else fixed — this is a quick way to see, first-hand, the same lead-time and country effects documented in Section 4.

9. Strengths of the Project

- **Complete, end-to-end pipeline** — from raw data to a deployed, usable application, covering every standard stage of a real ML workflow.
- **Thoughtful, feature-specific encoding** — label, ordinal, and frequency encoding are each matched to the nature of the column instead of a one-size-fits-all approach, which controls dimensionality (especially for the high-cardinality country column) while preserving signal.
- **Rigorous model comparison** — thirteen algorithms, including two ensembling strategies (voting and stacking), evaluated on eight different metrics rather than accuracy alone.
- **Honest overfitting check** — comparing train vs. test accuracy surfaced that Random Forest, Extra Trees, and Decision Tree overfit heavily (~99.8% train accuracy), guiding the choice toward the better-generalizing CatBoost.
- **Systematic feature selection** — top-N feature counts were empirically compared rather than assumed, landing on a 20-feature subset that balances performance with simplicity.
- **Well-tuned final model** — RandomizedSearchCV hyperparameter tuning plus feature selection produced a strong, well-generalized ROC AUC of ~0.92.
- **Actionable business insights** — the lead-time, seasonality, and country findings are concrete and directly usable by a hotel's revenue-management team.
- **Functional, user-friendly deployment** — a clean, organized Streamlit interface that mirrors the exact training-time preprocessing at inference time, avoiding train/serve skew.

10. Weaknesses & Limitations

- **Class imbalance is not explicitly addressed** — with ~27% of bookings canceled, no resampling (SMOTE/class weighting) appears to have been applied; this likely caps recall on the cancellation class (~69%), meaning roughly 3 in 10 cancellations still go undetected.
- **Hidden defaults reduce real-world accuracy** — the app silently hard-codes meal plan to "BB" and arrival month to "January" instead of exposing them as inputs, even though the EDA itself shows arrival month is one of the strongest seasonal cancellation signals. A user predicting a July booking is unknowingly scored as if it were a January booking.
- **Free-text country field is error-prone** — a plain text box (default "PRT") with no validation or dropdown means typos or invalid ISO codes silently fall back to a frequency of 0, which can distort the prediction without any warning to the user.
- **Several behavioral inputs are frozen to defaults** — adults/children/babies, repeated-guest status, and previous non-canceled bookings are fixed rather than user-editable, even though repeated-guest status and party composition are meaningful predictors in the underlying data.
- **No input validation or explanation layer** — the app returns a bare probability without showing which features pushed the prediction toward cancellation (e.g. no SHAP/feature-contribution view), which limits trust and interpretability for hotel staff.
- **Deployment is local-only** — the app is designed to run via `streamlit run app.py` on a local machine; there is no evidence of cloud hosting, authentication, or logging for production use.

- **Encoders trained on a fixed vocabulary** — label encoders will raise an error (rather than degrade gracefully) if a genuinely new category value appears at inference time (e.g. a room type not seen in training).

11. Recommended Improvements

- Expose **arrival month** and **meal plan** as real inputs in the UI instead of hard-coded defaults, given how strong the seasonal signal is in the EDA.
- Replace the free-text country field with a validated dropdown or searchable selector populated from the known country list used during training.
- Address class imbalance directly — e.g. class-weighting, threshold tuning, or SMOTE/undersampling — and report metrics at the adjusted decision threshold.
- Add a model-explanation panel (SHAP values, as the project's own "Future Improvements" section already proposes) so staff can see *why* a booking is flagged as high-risk.
- Wrap encoders/imputers with graceful fallbacks for unseen categories instead of raising exceptions.
- Add basic input validation (e.g. arrival date sanity checks, ADR/lead-time ranges) and inline help text for each field.
- Move deployment to a persistent host (Streamlit Community Cloud, Hugging Face Spaces, or a containerized cloud service) with basic usage logging for monitoring drift over time.
- Introduce periodic retraining as new booking data accumulates, so the model does not go stale relative to changing demand patterns.

12. Conclusion

This project successfully demonstrates a full machine-learning lifecycle applied to a genuine business problem: predicting hotel booking cancellations. The combination of careful, feature-specific preprocessing, a broad and honestly-evaluated model comparison, and a working interactive deployment makes it a strong, well-rounded portfolio piece. The clearest next steps are tightening the gap between the trained pipeline and the deployed app — particularly around the fields currently hard-coded to defaults — and adding interpretability and imbalance-handling to turn a solid predictive model into an even more trustworthy decision-support tool for hotel operations.

Document generated as project documentation for the NTI Machine Learning for Data Analysis graduation project — Hotel Booking Cancellation Prediction.